CS432 : Databases

Assignment 2 (Track 1)

Module B

Group : NULL VALUE

Aakash Venkatesan(23110002), BHC Karthikeya (23110070), Vyomika Vasireddy(23110363)

D A S K R Manognya(24110097), Kaushik(22110113)

Schema Design

The Laundry Management System database is structured into two distinct layers: core system tables that handle authentication and user management, and project-specific tables that model the laundry domain.

This separation ensures that credentials are never duplicated across tables, maintaining strict data Integrity.

Core System Tables

The **Users** table serves as the single source of truth for login credentials. It stores UserID, Username, Password, Role, and CreatedAt. Every authenticated entity, whether a customer or staff member, has exactly one entry in this table. No other table stores passwords or login credentials.

• UserID links the Users table to both Customer and Staff tables via foreign keys.
• Role field enforces RBAC at the database level (values: admin, staff, customer).
• Credentials are stored once and referenced, never copied into project tables.

Project-Specific Tables

The project tables model the laundry domain and reference the Users table for identity:

Table	Primary Key	Links To	Purpose
customer	MemberID	users (UserID)	Stores customer profile details
staff	StaffID	users (UserID)	Stores staff member details
orders	OrderID	customer (MemberID)	Tracks laundry orders

garment	GarmentID	orders, item, service	Links order items to services
item	ItemID	-	Defines clothing item types
service	ServiceID	-	Defines laundry service types
pickup	OrderID	orders, staff	Tracks order pickup logistics
delivery	OrderID	orders, staff	Tracks order delivery logistics
bill	BillID	orders	Billing record per order
payment	PaymentID	bill	Payment transaction per bill
feedback	FeedbackID	orders	Customer feedback per order

Cascade rules (ON DELETE CASCADE, ON UPDATE CASCADE) are applied on all foreign keys to automatically maintain referential integrity when parent records are modified or deleted.

Member Lifecycle Integrity

When a customer or staff member is deleted through the admin UI, the backend performs a safe deletion sequence:

- it first verifies no dependent records exist (e.g., active orders for a customer, or active pickup/delivery assignments for staff), then deletes the profile record, and finally removes the corresponding Users entry only if it is not linked to any other entity. This prevents orphaned credentials while maintaining referential integrity.

Security - session validation and RBAC

Authentication Flow

Every request to a protected endpoint must carry a valid JWT (JSON Web Token) in the Authorization header. The token is issued at login and expires after 1 hour.

Step	Action
1	User submits username, password, role, and admin code (if admin) via POST /login
2	Backend queries Users table, verifies password, checks role matches
3	If admin, verifies ADMIN_SECRET code from request against environment variable

4	JWT token signed with JWT_SECRET, returned to client
5	Client stores token in localStorage, attaches as Bearer token on every request
6	verifyToken middleware decodes and validates token on every protected route
7	If token invalid or expired, 401 Unauthorized is returned immediately

Role-Based Access Control (RBAC)

The allowRoles() middleware is applied to every route that requires role-specific access. It reads the role from the decoded JWT token and rejects the request if the role does not match the allowed roles for that endpoint.

Endpoint	Method	Admin	Staff	Customer
/dashboard	GET	Full	Own tasks	Own profile
/orders	GET	All	Denied	Own orders
/orders	POST	Yes	Denied	Yes (own)
/orders/:id	PUT	Any	Denied	Own only
/orders/:id	DELETE	Yes	Denied	Denied
/customers	POST	Yes	Denied	Denied
/customers/:id	PUT	Yes	Denied	Denied
/customers/:id	DELETE	Yes	Denied	Denied
/staff	POST	Yes	Denied	Denied
/staff/:id	PUT	Yes	Denied	Denied
/staff/:id	DELETE	Yes	Denied	Denied
/me/customer	PUT	Denied	Denied	Own only
/logs	GET	Yes	Denied	Denied

Audit Logging

Every data-modifying API call triggers a log entry written to logs/audit.log. Each entry records the timestamp, username, role, action performed, and outcome status. This allows any unauthorized or direct

database modification (bypassing the API) to be immediately identified during a log review — such entries will be absent from the log, while all legitimate API operations will appear with their full context.

Sample audit.log entries:

```
[2026-03-22T09:18:45.812Z] user=admin role=admin action=Created customer 100 status=SUCCESS
[2026-03-22T09:19:00.542Z] user=vyom role=unknown action=Login failed - wrong password status=FAILED
[2026-03-22T09:20:14.216Z] user=vyom role=customer action=Login successful status=SUCCESS
[2026-03-22T09:23:59.578Z] user=rakeshjoshi role=staff action=Login successful status=SUCCESS
[2026-03-22T09:24:25.490Z] user=rajeshkumar role=staff action=Login successful status=SUCCESS
[2026-03-22T09:51:30.318Z] user=admin role=admin action=Login successful status=SUCCESS
[2026-03-22T10:12:13.072Z] user=rakeshjoshi role=staff action=Login successful status=SUCCESS
[2026-03-22T10:15:01.384Z] user=rajeshkumar role=staff action=Login successful status=SUCCESS
[2026-03-22T10:15:20.467Z] user=gauravbansal role=staff action=Login successful status=SUCCESS
[2026-03-22T10:15:32.641Z] user=rakeshjoshi role=staff action=Login successful status=SUCCESS
[2026-03-22T11:13:24.040Z] user=admin role=admin action=Login successful status=SUCCESS
[2026-03-22T11:14:54.607Z] user=admin role=admin action=Created customer 111 status=SUCCESS
[2026-03-22T11:17:22.932Z] user=admin role=admin action=Login successful status=SUCCESS
[2026-03-22T11:18:56.407Z] user=admin role=admin action=Created customer 112 status=SUCCESS
[2026-03-22T11:19:11.776Z] user=vyomi role=customer action=Login successful status=SUCCESS
[2026-03-22T11:19:26.187Z] user=vyomi role=customer action=Updated own customer profile status=SUCCESS
[2026-03-22T11:19:45.010Z] user=aarav1 role=customer action=Login successful status=SUCCESS
[2026-03-22T11:20:02.803Z] user=rakeshjoshi role=staff action=Login successful status=SUCCESS
[2026-03-22T11:20:23.245Z] user=admin role=admin action=Login successful status=SUCCESS
[2026-03-22T11:21:44.785Z] user=admin role=admin action=Deleted customer 111 status=SUCCESS
[2026-03-22T11:22:34.090Z] user=admin role=unknown action=Login failed - invalid admin code status=FAILED
```

Indexing Strategy

SQL indexes were applied to columns that appear in WHERE, JOIN ON, and ORDER BY clauses of the most frequently executed queries in the application. Without indexes, MySQL performs a full table scan, reading every row to find matching records. With indexes, MySQL uses a B+ Tree structure to locate records directly, drastically reducing the number of rows examined.

Identified Bottleneck Queries

Query	Endpoint	Frequency
SELECT * FROM customer WHERE UserID = ?	/dashboard (customer login)	Every login
SELECT o.*, c.Name FROM orders o LEFT JOIN customer c ON ...	/dashboard (admin)	Every admin load
SELECT * FROM pickup WHERE StaffID = ?	/dashboard (staff login)	Every staff login
SELECT * FROM orders WHERE OrderStatus = ?	/orders (filter)	Every filter action
SELECT COUNT(*) FROM orders WHERE OrderStatus = ?	/orders (count)	Every status count

Indexes Applied

Index Name	Table	Column(s)	Targets
idx_customer_userid_name_email	customer	UserID, Name, Email	WHERE UserID = ? and JOIN queries needing Name/Email — composite for login and profile
idx_orders_memberid	orders	MemberID	JOIN ON MemberID in admin dashboard query
idx_orders_status	orders	OrderStatus	WHERE OrderStatus = ? for filter and COUNT queries — covering index
idx_pickup_staffid	pickup	StaffID	WHERE StaffID = ? on staff dashboard pickup view
idx_orders_orderdate	orders	OrderDate	ORDER BY OrderDate for recent orders and date-based reporting

SQL Commands

```
CREATE INDEX idx_customer_userid_name_email ON customer(UserID, Name, Email);
CREATE INDEX idx_orders_memberid               ON orders(MemberID);
CREATE INDEX idx_orders_status          ON orders(OrderStatus);
CREATE INDEX idx_pickup_staffid         ON pickup(StaffID);
CREATE INDEX idx_orders_orderdate       ON orders(OrderDate);
```

Performance Benchmarking

Query execution times were measured using MySQL's EXPLAIN ANALYZE statement, which provides actual execution time, rows examined, and the access method used (full table scan vs index lookup). The database was populated with a larger synthetic dataset to produce meaningful performance differences.

EXPLAIN ANALYZE - Before Indexing

Before indexes were applied, all queries used a Full Table Scan (type: ALL), meaning MySQL read every row in the table to find matches:

Query	SQL	Scan Type	Rows Examined	Execution Time
Q1 - Customer by UserID	SELECT * FROM customer WHERE UserID = 3	Full Table Scan	21 rows	0.0600 ms

Q2 - Orders JOIN Customer	SELECT o.OrderID, c.Name FROM orders o LEFT JOIN customer c ON ..	Nested Loop (PRIMARY key)	20 rows	0.0852 ms
Q3 - Pickup by StaffID	SELECT * FROM pickup WHERE StaffID = 1	Index Lookup (existing)	0 rows	0.0340 ms
Q4 - Orders by Status	SELECT * FROM orders WHERE OrderStatus = 'Pending'	Full Table Scan	20 rows	0.0619 ms
Q5 - COUNT by Status	SELECT COUNT(*) FROM orders WHERE OrderStatus = 'Pending'	Full Table Scan	20 rows	0.0530 ms

EXPLAIN ANALYZE - After Indexing

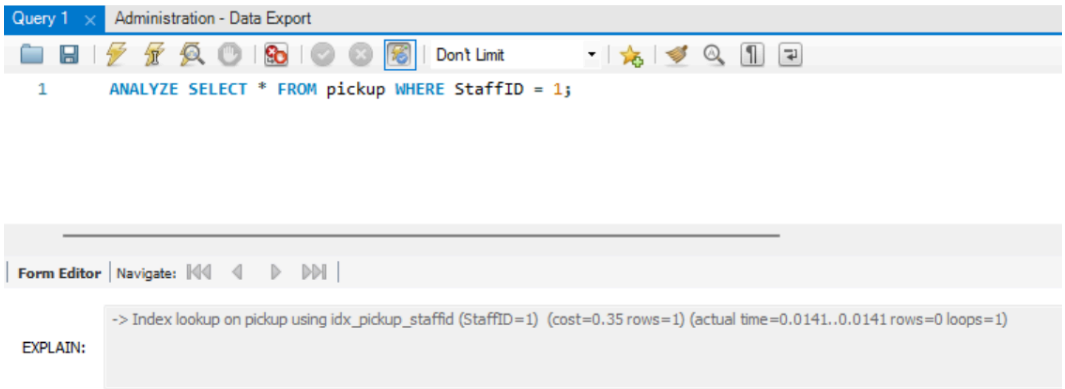
After indexes were applied, Q1, Q4, and Q5 switched to Index Lookup. Q5 additionally benefited from a Covering Index, meaning MySQL retrieved all required data directly from the index without touching the table at all.

Query	Scan Type	Rows Examined	Execution Time	Index Used
Q1 - Customer by UserID	Index Lookup	1 row	0.0347 ms	idx_customer_userid_name_email
Q2 - Orders JOIN Customer	Nested Loop (PRIMARY)	20 rows	0.1390 ms	PRIMARY (no change)
Q3 - Pickup by StaffID	Index Lookup	0 rows	0.0141 ms	idx_pickup_staffid
Q4 - Orders by Status	Index Lookup	1 row	0.0280 ms	idx_orders_status
Q5 - COUNT by Status	Covering Index Lookup	1 row	0.0185 ms	idx_orders_status (covering)

Example Query 3 : before and after

Before-

After-



Summary

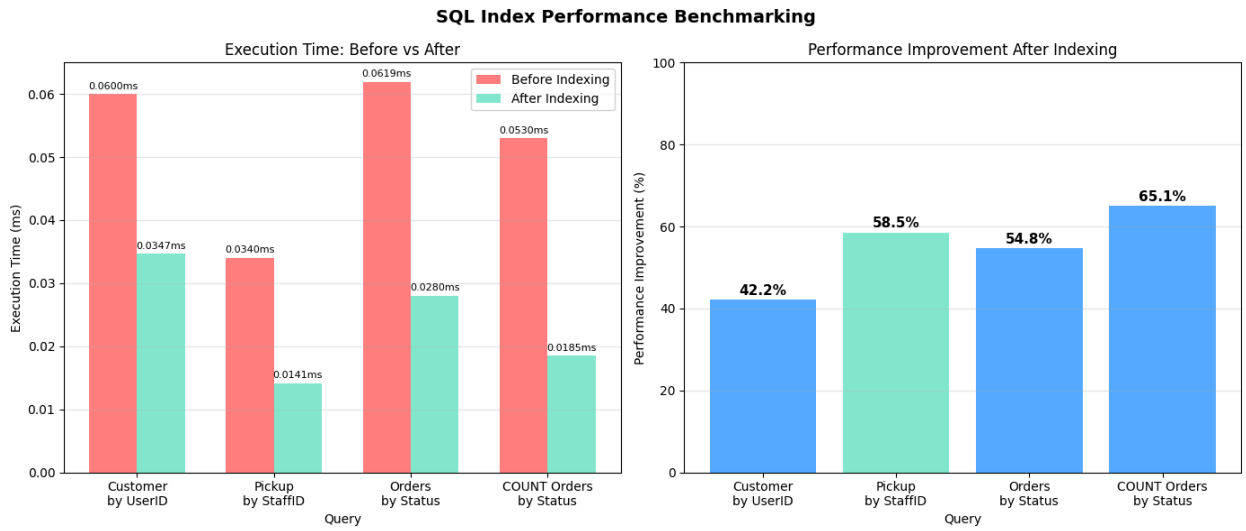
Comparison

Query	Before (ms)	After (ms)	Change	Improvement
Q1 - Customer by UserID	0.0600	0.0347	Faster	42.2%
Q2 - Orders JOIN Customer	0.0852	0.1390	Slower	N/A (small dataset)
Q3 - Pickup by StaffID	0.0340	0.0141	Faster	58.5%
Q4 - Orders by Status	0.0619	0.0280	Faster	54.8%
Q5 - COUNT by Status	0.0530	0.0185	Faster	65.1%

[Q2 (Orders JOIN Customer) shows a slight execution time increase (0.0852ms -> 0.139ms) due to the very small dataset size (20 rows). With larger datasets, the idx_orders_memberid index would significantly reduce join cost by enabling direct key lookups instead of full scans on the customer side]

Benchmarking Graph

The graph below visualises execution times before and after indexing, and the percentage improvement for Q1, Q4, and Q5, the three queries that showed clear measurable improvement. Q2 is excluded from the graph due to dataset size anomaly (explained above).



Video Demonstration

A 3-5 minute screen-capture video has been recorded demonstrating the fully integrated local system. The video covers:

- UI & API Functionality - Navigation of the web UI, Member Portfolio, and CRUD operations on project tables.
- RBAC Enforcement - Admin login demonstrating full access, followed by Customer and Staff logins demonstrating restricted access.
- Security Logging - Audit log displayed showing valid API operations, with explanation of how unauthorized direct DB modifications would be identified.

Video Link:

https://drive.google.com/drive/folders/1CpwNmCknEbFtc_eMcfc15k2YI8kEEA5F?usp=sharing

Team members names and contributions:

Aakash Venkatesan (23110002) - Module B

Vyomika Vasireddy (23110363) - Module B

BHC Karthikeya (23110070) - Module A

D A S K R Manognya (24110097) - Module B

Kaushik (22110113) - Module A